

Hybrid Sparse–Dense Retrieval for Privacy-Constrained Incident Resolution: Design and Evaluation of a Locally-Hosted Retrieval-Augmented Assistant

Ayoub Aarab

*Master Data Science — End-of-Year Project
Ecole Normale Supérieure, Martil*

August 4, 2026

Abstract

Retrieval-augmented generation (RAG) grounds a language model’s output in an external corpus, but its quality is bounded by the retrieval stage. In operational IT incident management, queries are heterogeneous: some are paraphrased descriptions of a symptom, while others are exact identifiers — incident numbers, error codes, function and table names. Dense embedding retrieval handles the former well and the latter poorly, since semantically near-identical identifier strings are not separable in embedding space.

This work designs and evaluates an incident-resolution assistant built on a hybrid retriever that fuses a dense sentence-embedding index with a sparse Okapi BM25 index using Reciprocal Rank Fusion (RRF). The system operates under a strict privacy constraint: all inference — embeddings and generation — executes on locally-hosted models, with no external API calls. The retriever is evaluated on a labeled benchmark of 272 queries in four stylistic classes over a corpus of 68 incident reports spanning nine technical domains, using Recall@ k , Mean Reciprocal Rank, and nDCG@5.

Hybrid retrieval attains Recall@5 = 1.000 and MRR = 0.987, against 0.768 and 0.678 for dense-only retrieval. A per-class analysis localizes the improvement: on exact-identifier queries, dense-only Recall@5 is 0.221 while the hybrid attains 1.000, confirming that the fusion supplies precisely the lexical precision the dense index lacks. Ablations show retrieval quality is invariant to chunk size over [400, 1000] characters and saturates at $k \geq 3$, a result used to bound the generation context. Error analysis records a 0.0% failure rate over the benchmark. The retrieval component is embedded in a grounded generation pipeline with intent-based abstention and adversarial-input defenses, and in an automated report-synthesis mechanism that maps a resolved dialogue onto a validated structured schema.

Beyond ranking, the work addresses the gap between what a small local model is instructed to do and what it does. A source-selection stage admits the empty

set, so questions the corpus does not document yield no citation rather than a spurious one; a post-conditioning stage removes artifacts unattested by the retrieved evidence, bounds self-reported confidence by the answer’s actual content, and annotates irreversible operations according to whether a report documents them. Layered validation — 269 component tests, a 69-report retrieval sweep, and a 16-case behavioral suite against the deployed model — accompanies the retrieval benchmark. A controlled comparison against a model of $2.7\times$ the parameter count finds identical correctness (15/15 reports, 60/60 property checks) at 43% higher latency, supporting the central claim that under a local-inference constraint the decisive variable is context quality rather than model scale.

Contents

1	Introduction	5
1.1	Problem Setting	5
1.2	Deployment Constraint: Local Inference	5
1.3	Objectives and Structure	5
2	Background	6
2.1	Retrieval-Augmented Generation	6
2.2	Sparse Retrieval: Okapi BM25	6
2.3	Dense Retrieval	6
2.4	Rank Fusion	7
3	System Architecture	7
3.1	Design Overview	7
3.2	Structured Report Schema as a Shared Contract	8
3.3	Model Provider Abstraction	8
3.4	Deployment	9
4	Request Pipeline and Control Flow	9
4.1	Staged Execution with Early Termination	9
4.2	Corpus Maintenance Workflow	10
5	Corpus, Representation, and Evaluation Protocol	11
5.1	Corpus	11
5.2	Document Representation	13
5.3	Chunking and Indexing	14
5.4	Evaluation Protocol	14
6	Retrieval Method	15
6.1	Components	15
6.2	Fusion	15
6.3	Source Selection	16
7	Experimental Results	18
7.1	Aggregate Retrieval Quality	18
7.2	Per-Class Analysis	18
7.3	Ablations	19
7.4	Error Analysis	20
8	Grounded Generation	20
8.1	Conditioning and Output Structure	20
8.2	Abstention Under Insufficient Evidence	20
8.3	Adversarial Input	21
8.4	Feedback	21

9	Post-Conditioning: Reconciling Answers with Evidence	22
9.1	Artifact Grounding	22
9.2	Confidence Calibration	22
9.3	Irreversible Operations	23
9.4	Determinism	23
10	Report Synthesis	24
11	Efficiency	25
11.1	Caching	25
11.2	Measured Behavior	26
11.3	Concurrency	26
12	Validation Methodology	27
12.1	Component Tests	28
12.2	Corpus Sweep	28
12.3	Behavioral Suite	28
12.4	Model Selection as a Measured Decision	29
13	Discussion	29
13.1	Interpretation	29
13.2	Limitations	30
13.3	Future Work	30
13.4	Conclusion	31

1 Introduction

1.1 Problem Setting

Incident management in large-scale IT operations produces a corpus of structured post-mortem reports: each documents a symptom, a diagnosed root cause, a sequence of resolution steps, and supporting artifacts such as queries, commands, or configuration. This corpus is institutional knowledge. Its operational value depends entirely on whether an engineer facing a new incident can retrieve the relevant prior cases quickly and reliably.

Retrieval-augmented generation [4] is the natural formulation: retrieve the most relevant prior reports, condition a language model on them, and synthesize a grounded resolution. The scientific difficulty lies not in the generation stage but in retrieval. Incident queries are stylistically heterogeneous, and the two dominant retrieval paradigms fail on disjoint subsets of them:

- **Dense retrieval** embeds queries and documents into a shared metric space and ranks by cosine similarity. It captures paraphrase and semantic proximity, but identifiers such as INC0012017 and INC0012018 are lexically near-identical and semantically indistinguishable; the embedding space does not separate them.
- **Sparse retrieval** (Okapi BM25 [1]) ranks by weighted term overlap. It matches identifiers and rare technical tokens exactly, but is blind to synonymy and paraphrase.

An incident assistant must serve both query classes. This motivates a hybrid retriever, and it raises the central empirical question of this work: *how much does hybridization actually contribute, and on which queries does that contribution arise?*

1.2 Deployment Constraint: Local Inference

The system is subject to a hard privacy requirement: incident reports contain sensitive operational detail and must not be transmitted to third-party inference APIs. Consequently every model — the sentence encoder and the generative model — is self-hosted, and the pipeline performs no external network calls at query time.

This constraint is not incidental; it conditions the design. It excludes frontier hosted models, imposing a lower generation-quality ceiling and a hardware-bound latency floor. It therefore shifts the engineering leverage away from model scale and toward *context quality*: what is retrieved, and how it is presented to the model. This shift is the technical thesis of the work.

1.3 Objectives and Structure

The objectives are: (i) to design a retrieval pipeline that serves both semantic and lexical query classes under local-inference constraints; (ii) to quantify its behavior on a labeled benchmark with standard information-retrieval metrics and ablations; (iii) to embed it in a generation pipeline that abstains rather than fabricates when evidence is insufficient, and to enforce that property by post-conditioning rather than by instruction alone; (iv) to close the documentation loop by synthesizing new structured reports from resolved dialogues; and (v) to establish, by measurement, that the resulting system’s quality derives from context construction rather than from generative model scale.

Section 2 reviews the relevant retrieval background. Section 3 presents the system architecture and Section 4 the request pipeline and corpus workflow. Section 5 describes the corpus, its representation, and the evaluation protocol. Section 6 formalizes the retrieval method, including the source selection that permits abstention. Section 7 reports retrieval results. Sections 8–9 cover grounded generation and the post-conditioning that enforces its guarantees, and Section 10 report synthesis. Sections 11–12 report efficiency and the validation methodology, including the measured model-selection decision. Section 13 discusses limitations.

2 Background

2.1 Retrieval-Augmented Generation

RAG [4] conditions a generative model on documents retrieved from a non-parametric memory. Formally, given a query q , a retriever selects a set $D_q \subset \mathcal{D}$ from a corpus $\mathcal{D}$, and a generator produces an answer $a \sim p(a \mid q, D_q)$. Two properties make this attractive for incident resolution: the model’s output can be traced to cited evidence, and the corpus can be updated without retraining. The critical dependency is that $p(a \mid q, D_q)$ can only be as good as D_q : retrieval failure is unrecoverable downstream, since a model conditioned on irrelevant context either hallucinates or abstains.

2.2 Sparse Retrieval: Okapi BM25

BM25 [1] scores a document d against query q as

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}, \quad (1)$$

where $f(t, d)$ is the frequency of term t in d , $|d|$ the document length, avgdl the mean document length, and $\text{IDF}(t)$ the inverse document frequency. The parameter k_1 controls term-frequency saturation and b the strength of length normalization. The inverse-document-frequency factor assigns high weight to rare terms — exactly the regime of incident identifiers, error codes, and function names.

2.3 Dense Retrieval

Dense retrieval encodes text with a transformer bi-encoder trained so that semantically related pairs are close under cosine similarity [3]. Ranking is by

$$s_{\text{dense}}(q, d) = \frac{\phi(q) \cdot \phi(d)}{\|\phi(q)\| \|\phi(d)\|}, \quad (2)$$

with ϕ the encoder. This generalizes across surface form, but its inductive bias — mapping semantically similar text to nearby vectors — is precisely what makes it unable to discriminate between distinct identifiers that are semantically equivalent (all are “an incident number”).

2.4 Rank Fusion

Combining Eq. (1) and Eq. (2) by score arithmetic is ill-posed: the two are on incompatible, unnormalized scales, and any weighting requires per-corpus calibration. Reciprocal Rank Fusion [2] avoids this by discarding magnitudes and using only rank position:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}, \quad (3)$$

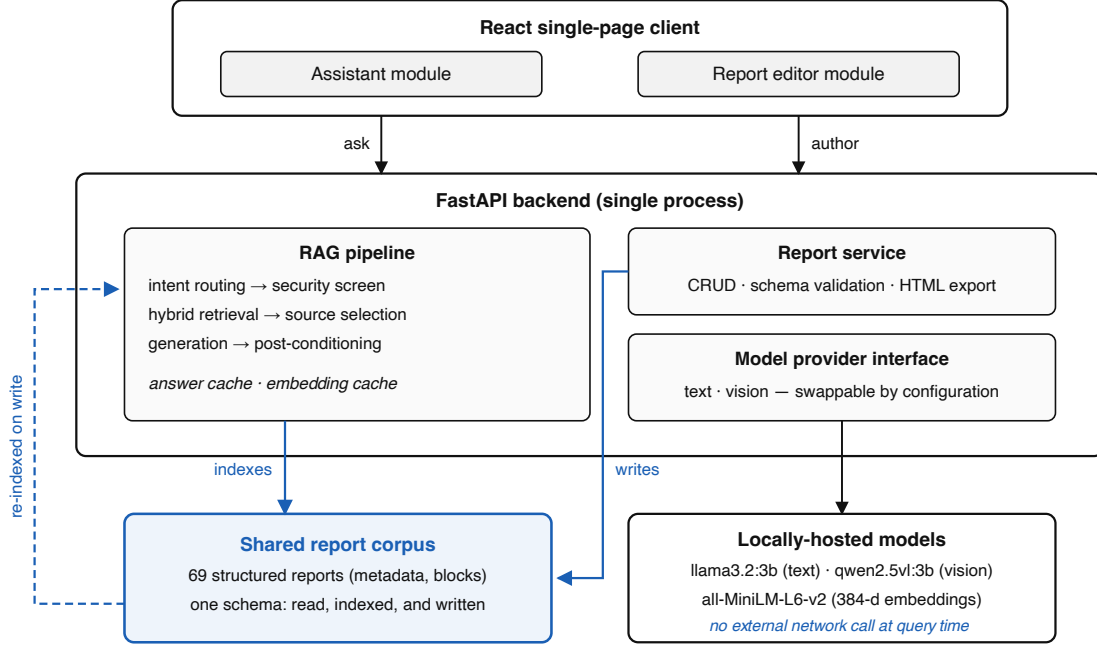
where R is the set of ranked lists and k a smoothing constant that damps the influence of top ranks. RRF is scale-free and effectively parameter-free, which is why it is adopted here.

3 System Architecture

3.1 Design Overview

The system is a single application exposing two functional modules over a shared corpus: an *assistant* that answers incident questions by retrieval-augmented generation, and a *report editor* that authors and maintains the structured reports that constitute the corpus. Both are served by one backend process and one web client.

The implementation is a modular monolith: a FastAPI backend with separated internal packages (retrieval and generation; report persistence; a shared schema and model-provider abstraction) and a React single-page client. A distributed decomposition is unwarranted — the modules share a corpus, target a single host, and have no independent scaling requirement — and would introduce network and operational overhead without benefit.



The corpus is the only coupling between the two modules — and the reason a resolved dialogue can become retrievable knowledge.

Figure 1: System architecture: the web client (assistant and report-editor modules) over a FastAPI backend hosting the RAG pipeline, the report service, and the model-provider abstraction, against locally-hosted models and the shared report corpus.

3.2 Structured Report Schema as a Shared Contract

Reports are represented by a typed schema, `Report = (metadata, blocks)`, defined once as Pydantic models and mirrored in the client’s type system. The `metadata` record carries identifying attributes (identifier, title, category, subcategory, reporter, date). The `blocks` field is a sequence over a discriminated union of content types: heading, paragraph, ordered/unordered list, code, table, image, and cross-referenced incident example.

This schema is the system’s central abstraction. It is simultaneously (i) the editor’s document model, (ii) the unit of ingestion for retrieval, and (iii) the output type of report synthesis (Section 10). A single validated contract across all three roles is what permits the assistant to both consume and produce corpus documents without translation layers.

3.3 Model Provider Abstraction

Generation is accessed through a provider interface exposing text and vision completion, with a locally-hosted implementation as the deployed backend. Decoupling the pipeline from a concrete model permits the generative model to be selected by configuration, and isolates the retrieval and orchestration logic from model-specific concerns. The deployed configuration uses a 3B-parameter instruction-tuned model for text and a 3B vision-language model for screenshot analysis; sentence embeddings use `all-MiniLM-L6-v2`, a 384-dimensional encoder.

3.4 Deployment

The application is containerized with a multi-stage build in which the client is compiled and served as static assets by the backend process, yielding a single-process deployment. Persistent state comprises the report corpus, the embedding-model cache, and a SQLite database holding dialogue history.

The container runs the application under an unprivileged account rather than as `root`. Because named volumes are created root-owned at mount time — which masks ownership assigned during the image build — an entry point assumes ownership of the mounted paths and then replaces itself with the application process under the unprivileged identity. Replacing rather than forking matters: the application becomes process 1 and therefore receives termination signals directly, so shutdown is orderly (measured at one second, exit status zero) instead of terminating by timeout, which would abandon in-flight response streams and risk a torn database write. A health probe reports readiness distinctly from liveness, allowing an orchestrator to restart a process that is running but unable to serve. Memory is bounded so that a fault in this container cannot exhaust the host it shares with the inference server.

4 Request Pipeline and Control Flow

The architecture of Section 3 describes what the components are. This section describes how a request moves through them, because the ordering of stages — not merely their presence — is what produces the behavior evaluated in Sections 7 and 9.

4.1 Staged Execution with Early Termination

A request is processed as an ordered sequence of stages, each of which may terminate the request and return. The design principle is that a stage which can rule out an answer should run before any stage that could fabricate one. Cheap, decisive checks therefore precede expensive, permissive ones.

1. **Intent routing.** The turn is classified as conversational, out-of-scope, under-specified, or diagnosable. Only the last proceeds; the others are answered directly, without retrieval or generation. This is the structural guarantee of Section 8: fabrication on non-diagnosable input is impossible because no generator is invoked.
2. **Adversarial screening.** Text is scanned for instruction-override and extraction patterns and enclosed in an untrusted-data delimiter (Section 8).
3. **Query construction.** The query is combined with prior dialogue turns and, where an image is supplied, with a vision-model description of it, forming the retrieval query.
4. **Hybrid retrieval.** Dense and sparse indices are queried and fused by Reciprocal Rank Fusion (Section 6).
5. **Source selection.** Candidates are de-duplicated by source document, re-scored by entity overlap with the query, and filtered by an absolute and a relative threshold (Section 6.3). An empty result here is a valid outcome and suppresses citation entirely.

6. **Conditioning.** Selected documents are supplied *in full* rather than as the matched fragment, together with any applicable stored corrections.
7. **Generation.** The model emits a structured object under constrained JSON decoding.
8. **Post-conditioning.** The parsed answer is reconciled against the evidence: unsupported citations and artifacts are removed, confidence is bounded by what the answer actually contains, and irreversible operations are annotated (Section 9).

Stages 1 and 5 are the two points at which the system declines to answer, and both precede generation. Stage 8 is the only stage that can weaken an answer after the model has produced it, and it never strengthens one — a deliberate asymmetry, since every check it applies is a check against over-claiming.

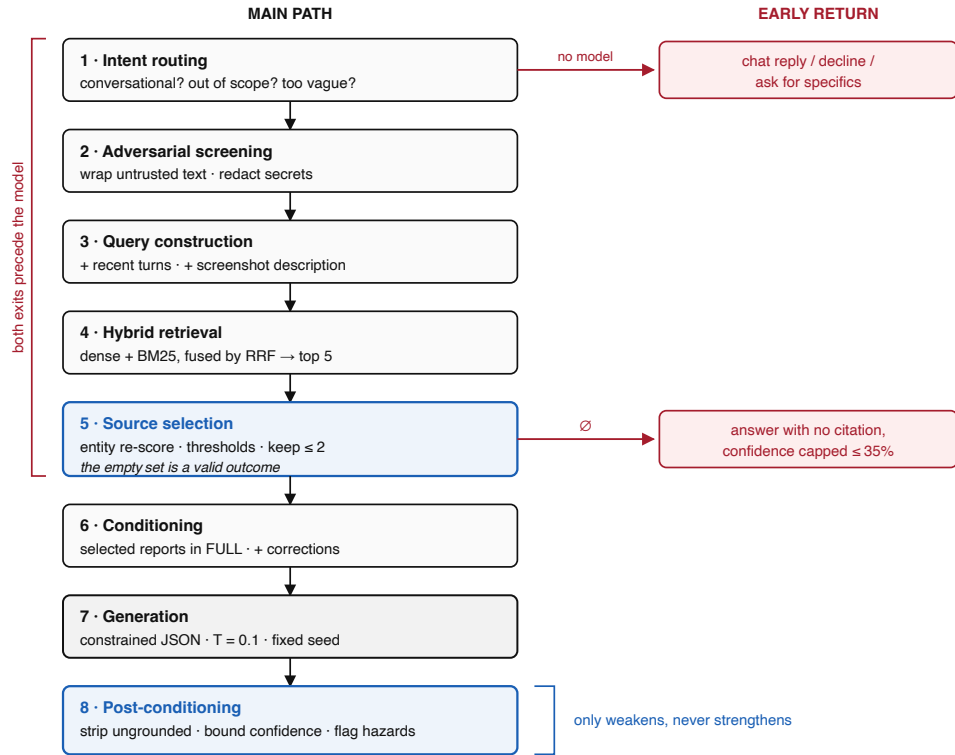


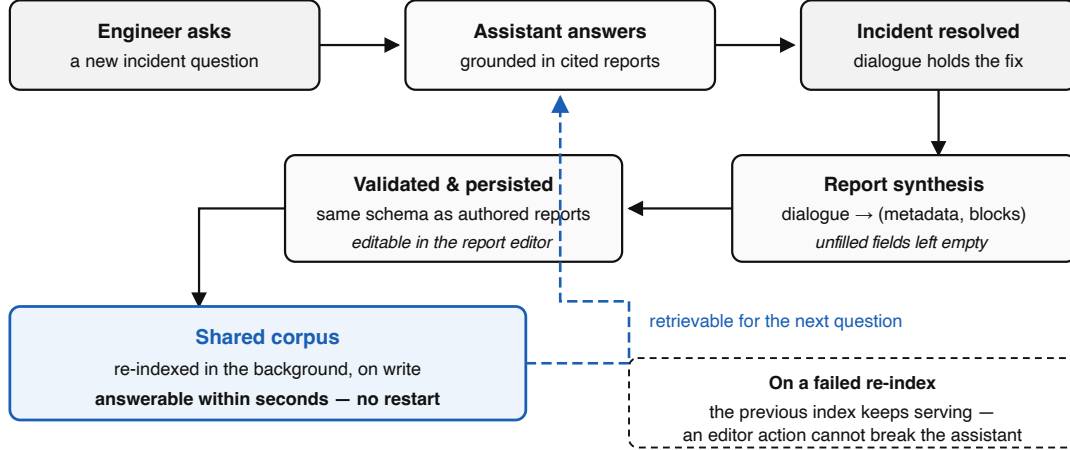
Figure 2: Request pipeline: intent routing and adversarial screening precede retrieval; hybrid retrieval feeds source selection, which may return the empty set; selected documents condition generation; post-conditioning reconciles the answer against the evidence before it reaches the client.

4.2 Corpus Maintenance Workflow

The two modules share the corpus, so a write by the editor must become visible to the assistant. Writing a report or deleting one triggers re-indexing as a background task: the response returns immediately and the index is rebuilt without the author waiting for embedding. The rebuilt index replaces the live one only on success, so a failed re-index leaves the previous working index serving requests rather than degrading the assistant.

as a side effect of an editor action. Cached answers are discarded at the same moment, since they were conditioned on evidence that has changed.

This closes the operational loop implied by Section 10: a resolution synthesized from a dialogue is answerable within seconds of being written, with no restart, so the corpus grows during an incident rather than after it.



The corpus grows during an incident rather than after it: each resolution becomes evidence for the next.

Figure 3: Corpus maintenance: a report written or deleted through the editor triggers background re-indexing and cache invalidation, making it retrievable by the assistant without a restart.

5 Corpus, Representation, and Evaluation Protocol

5.1 Corpus

The corpus comprises 68 incident reports spanning nine technical domains: networking, databases, authentication, infrastructure, frontend, messaging and queueing, continuous integration and delivery, monitoring, and security. Each report carries domain-characteristic failure signatures — HTTP status codes, message-broker error codes, container exit codes, TLS handshake alerts, DNS resolution failures.

The corpus is constructed to be discriminative rather than merely representative. It contains two families of near-miss pairs designed to defeat shallow matching:

- **Shared symptom, distinct root cause:** e.g. two incidents presenting as HTTP 503 from connection-pool exhaustion, one caused by a connection leak in an unrelated workload, the other by an undersized pool at peak concurrency.
- **Shared error code, distinct context:** e.g. container exit code 137 arising from node-level memory-pressure eviction versus a batch process exceeding its own memory limit.

A retriever that keyword-matches on 503 or 137 cannot separate these; correct ranking requires discriminating on context.

Table 1: Corpus statistics (source: evaluation notebook, cell 3).

Property	Value
Reports	68
Distinct categories	30
Median tokens per report	147.5
Token range per report	[103, 175]
Block-type frequency	
Paragraph	148
Heading	89
Code	72
List	68
Table	20
Incident example	12
Description box	8
Image	7

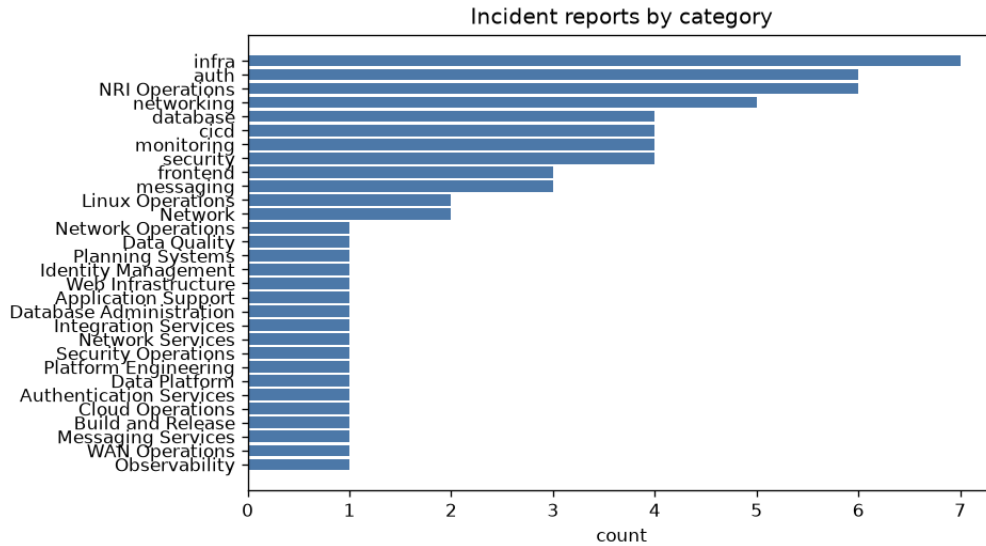


Figure 4: Category distribution over the corpus (source: notebook cell 3).

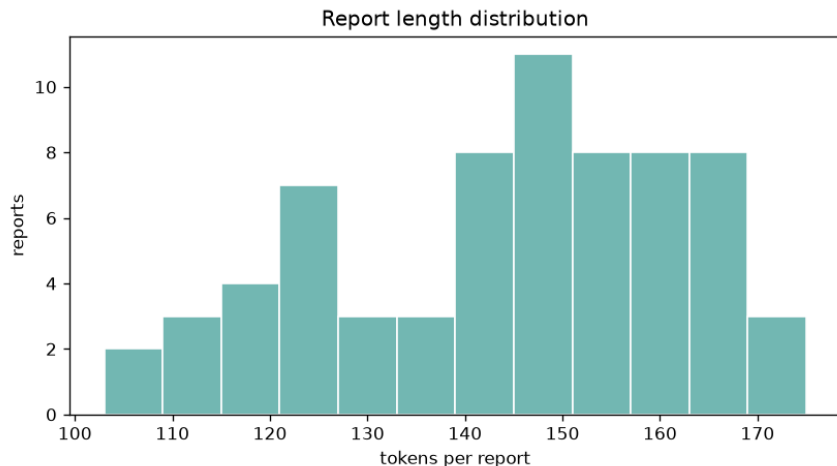


Figure 5: Report length distribution in tokens (source: notebook cell 3).

5.2 Document Representation

Reports are stored as structured documents, not free text, so the indexed representation must be derived by a projection $\pi : \text{Report} \rightarrow \Sigma^*$. The choice of π is consequential and easy to get wrong.

A naive projection serializes the document tree verbatim, emitting structural tokens — block identifiers, type discriminators, nesting keys, encoded image payloads — interleaved with content. The resulting string is dominated by tokens carrying no incident semantics. Both stages degrade: the encoder ϕ maps such strings into a region governed by shared structural vocabulary rather than incident content, collapsing inter-document distances; and BM25’s IDF weighting is diluted by high-frequency structural terms. Downstream, the generator is conditioned on text in which the actual symptom and resolution are a minority of tokens.

The system therefore defines π *schema-aware*: it dispatches on block type and emits only the semantic payload of each — heading text, symptom and root-cause prose with markup stripped, resolution list items, code content, table cells — while discarding identifiers, type tags, and binary data. Metadata is emitted as a leading header so that identifiers and category remain lexically searchable. This projection is a precondition for the retrieval results of Section 7: no amount of ranking refinement compensates for an uninformative representation.

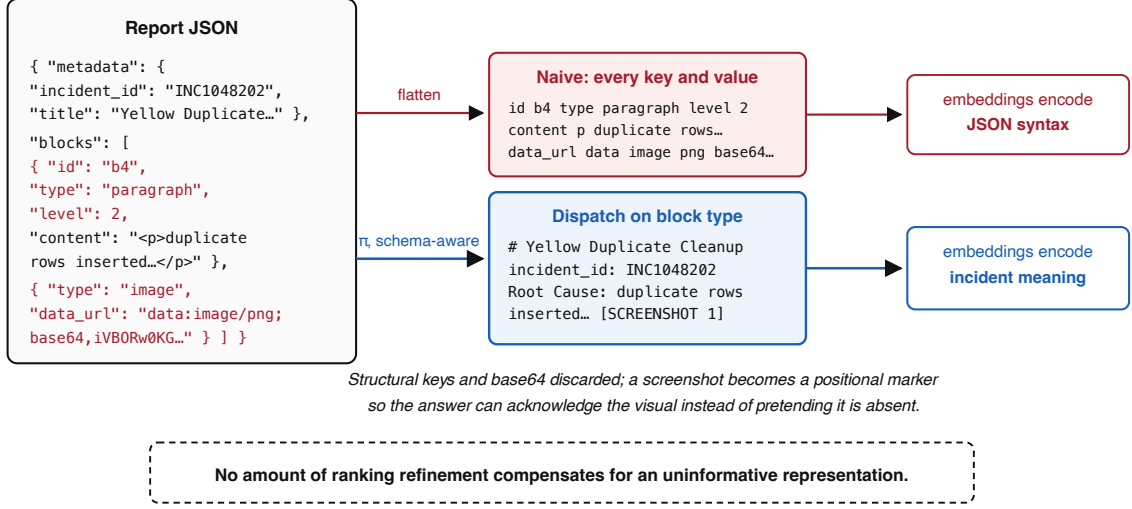


Figure 6: Document representation. The naive projection emits structural keys and base64 payloads, so the encoder embeds JSON syntax; the schema-aware projection π dispatches on block type and emits only incident content, replacing an inline screenshot with a positional marker.

5.3 Chunking and Indexing

Projected documents are segmented into overlapping character windows (700 characters, 120-character overlap) to bound the encoder’s input and to localize matching. Each chunk is encoded by ϕ into $\mathbb{R}^{384}$ and simultaneously tokenized into the BM25 index. Retrieval operates over chunks; results are aggregated to the report level by taking each source document’s maximum-scoring chunk, so a report is ranked by its single best-matching passage.

5.4 Evaluation Protocol

Evaluation requires query–document relevance labels. Absent production query logs, labels are constructed by inversion: for each report d , queries are synthesized that a user seeking d plausibly issues, with d as the gold document. Four stylistic classes are generated, chosen to span the semantic–lexical axis that motivates the hybrid:

- **title** — an interrogative form of the report title (semantic, high signal);
- **symptom** — the root-cause sentence (semantic, paraphrastic);
- **id** — the bare incident identifier (purely lexical);
- **keyword** — salient technical tokens such as function and table names (lexical, moderate).

This yields 272 labeled queries over the 68 reports. Stratifying by class is essential: an aggregate score would mask the fact that the two retrieval paradigms fail on *disjoint* query classes, which is the phenomenon under study.

Validity. The queries are synthetic and derived from the documents they retrieve, so the protocol measures recovery of a document from a paraphrase of its own content. Real queries are noisier, use vocabulary absent from the report, and may under-specify. The reported figures are therefore an upper bound on field performance. The protocol is nonetheless informative for its purpose here — it is a *comparative* instrument, and the dense/sparse/hybrid contrast is measured under identical conditions, so the relative ordering and the per-class localization of error are robust to this bias even where the absolute values are optimistic.

Metrics are Recall@ k (whether the gold report appears in the top k), Mean Reciprocal Rank, and nDCG@5, computed with a single relevant document per query.

6 Retrieval Method

6.1 Components

Three configurations are compared under identical chunking and identical corpus:

1. **Dense:** cosine ranking by Eq. (2) using `all-MiniLM-L6-v2`.
2. **Sparse:** BM25 by Eq. (1) with standard parameters $k_1 = 1.5$, $b = 0.75$. The index is implemented directly rather than by dependency, which keeps the retrieval stack self-contained and its behavior fully specified — relevant under a constraint that forbids external services. Tokenization preserves underscores, so composite identifiers such as `support_remove_opv_duplicate` remain single high-IDF terms rather than fragmenting into common subwords.
3. **Hybrid:** RRF over the dense and sparse rankings by Eq. (3), with $k = 60$.

6.2 Fusion

Each component produces a ranking over chunks. Documents scoring zero under BM25 are excluded from its list rather than appended arbitrarily, so that non-matching documents contribute no reciprocal-rank mass. The fused score of Eq. (3) is then aggregated per source report as described above, and the top K reports are returned.

TOP_K = 5 reports are retrieved. These are candidates, not citations: what reaches the generator and the user is determined by the selection stage below, which Section 7.3 shows can be restricted well below K without recall loss.

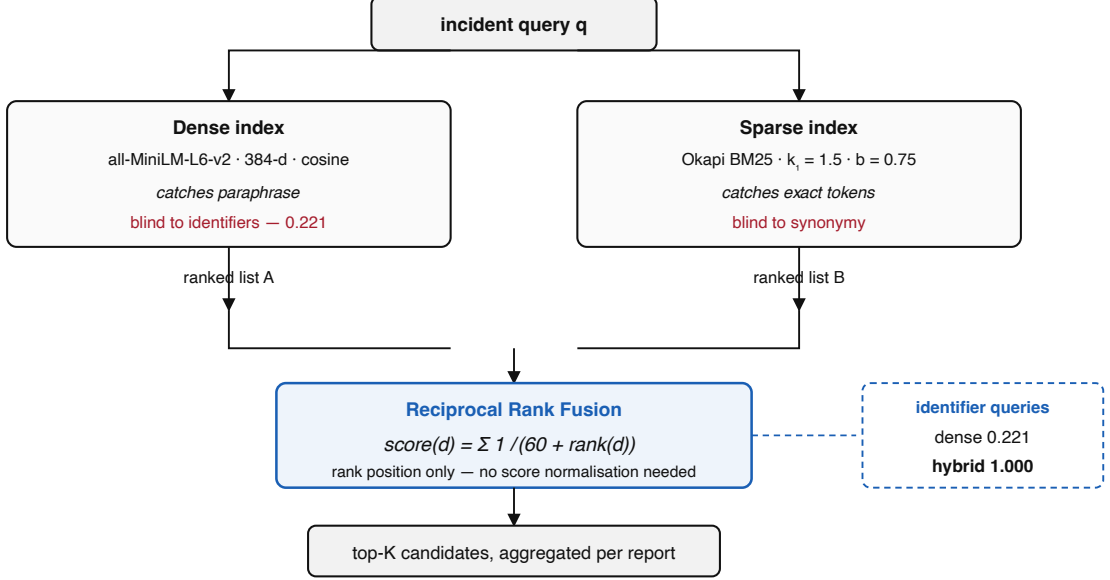


Figure 7: Retrieval architecture: a query is encoded and simultaneously matched against the dense index and the BM25 index; the two rankings are fused by RRF and aggregated per source report.

6.3 Source Selection

Ranking is not the same as relevance. RRF always returns its top- K candidates, ordered, whether or not any of them answers the query — the fusion score is a rank statistic, and a rank exists even when nothing matches. A pipeline that cites the top of that list unconditionally will attribute an answer to whichever document happened to rank first, which is worse than returning nothing: it lends a real incident’s authority to a guess.

A selection stage therefore intervenes between fusion and generation. Let H be the fused candidates and $E(q)$ the set of content terms in query q after stop-word removal. For a candidate h with retrieval score $s(h)$, define

$$\sigma(h) = s(h) + \lambda \cdot \frac{|E(q) \cap E(h)|}{|E(q)|}, \quad (4)$$

where $E(h)$ draws on the candidate’s title and body, with title matches weighted double, and λ scales the lexical contribution. The weighting reflects that RRF scores occupy a narrow range — they are reciprocals of small integers — so entity overlap, not fusion score, is what separates a document that *mentions* a term from one that *is about* it.

Candidates are de-duplicated by source report, ranked by σ , and admitted subject to two thresholds:

$$\sigma(h) \geq \tau_{\text{abs}} \quad \text{and} \quad \sigma(h) \geq \rho \cdot \max_{h' \in H} \sigma(h'), \quad (5)$$

with at most M reports retained. The relative threshold $\rho = 0.85$ prevents a single strong match from being padded with materially weaker ones. The absolute threshold $\tau_{\text{abs}} = 0.50$ is the one that permits the empty set, and it is what makes “nothing in the corpus documents this” an expressible outcome rather than an unreachable branch.

Its value is calibrated from the separation between question classes on this corpus, which is wide and unambiguous. Every incident question drawn from the corpus scores $\sigma \approx 1.04$; questions on subjects the corpus does not cover peak at 0.29 (Table 2). A threshold placed between them rejects the latter with margin on both sides while leaving every genuine match untouched — a property asserted directly by the test suite of Section 12, so that tightening τ_{abs} later cannot silently begin refusing questions the corpus does answer.

Table 2: Selection scores by question class. The threshold $\tau_{\text{abs}} = 0.50$ lies in the gap.

Class	Example	σ
Documented incident	“how to rollback lineIds”	1.042
Documented incident	“kafka consumer group lag growing”	1.042
Documented incident	“public API certificate expired”	1.042
Out of corpus	“configure the mainframe COBOL scheduler”	0.282
Out of corpus	“how do I tune a guitar”	0.288
Out of corpus	“how do I file my income taxes”	0.160
Out of corpus	“best recipe for sourdough bread”	0.029

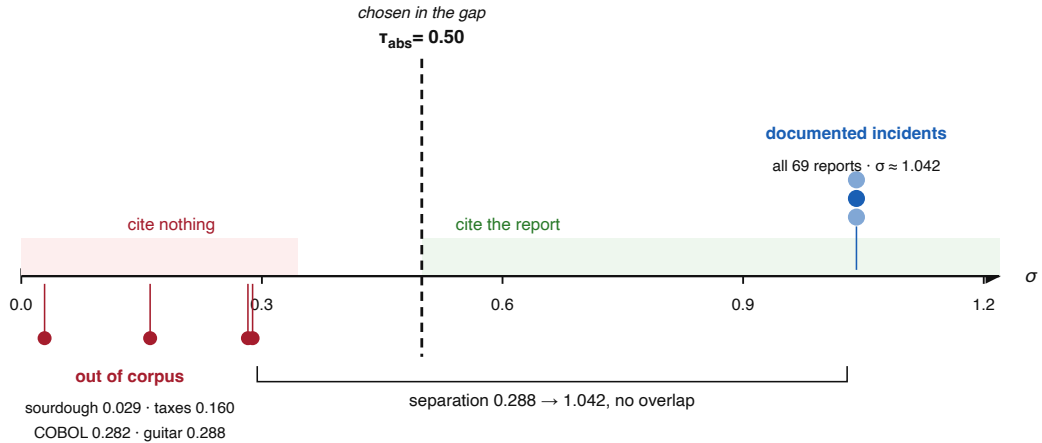


Figure 8: Source selection. Every documented incident question scores $\sigma \approx 1.042$; questions outside the corpus peak at 0.288. The absolute threshold is placed in the resulting gap, so a question the corpus does not answer yields no citation at all.

Selection caps the admitted set at $M = 2$ reports, and each is supplied to the generator *in full* rather than as the matched chunk. This inverts the usual context-budget trade-off: rather than many fragments from many documents, the generator receives the complete text of the few documents that genuinely match. A documented procedure is frequently a multi-step sequence — extract with a query, run a script, verify the result — and a chunk boundary that severs it yields an answer that is correct about the first step and silent about the rest.

7 Experimental Results

All quantitative results are produced by the evaluation notebook (`eval/evaluation.ipynb`); each table cites its source cell.

7.1 Aggregate Retrieval Quality

Table 3 reports aggregate performance over the 272-query benchmark. The hybrid dominates both components on every metric, attaining $\text{Recall@5} = 1.000$ and $\text{MRR} = 0.987$ against 0.768 and 0.678 for dense retrieval.

Table 3: Aggregate retrieval performance, 272 labeled queries (source: notebook cell 6).

Configuration	R@1	R@3	R@5	MRR	nDCG@5
Dense	0.629	0.713	0.768	0.678	0.700
Sparse (BM25)	0.779	0.982	0.989	0.872	0.902
Hybrid (RRF)	0.974	1.000	1.000	0.987	0.991

Two observations merit attention. First, sparse retrieval is the stronger single component on this corpus ($\text{Recall@5} = 0.989$ versus 0.768), which is consistent with a domain whose documents are dense in rare, discriminative technical tokens; this is a corpus property, not a general claim about BM25. Second, and more informative, the hybrid’s advantage over BM25 is concentrated at the top rank: Recall@1 improves from 0.779 to 0.974 while Recall@5 improves only from 0.989 to 1.000. The dense signal is therefore acting chiefly as a *re-ranking* prior — BM25 places the gold document in the candidate set, and the semantic signal promotes it to rank one. This is the mechanism by which the fusion earns its gain on this corpus.

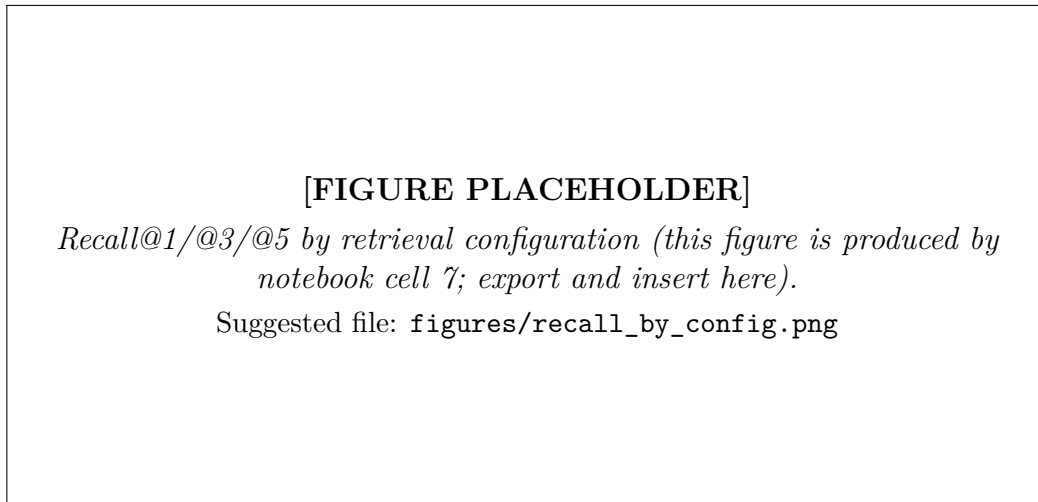


Figure 9: Recall@1/@3/@5 by retrieval configuration (this figure is produced by notebook cell 7; export and insert here).

7.2 Per-Class Analysis

The aggregate figures conceal the phenomenon of interest. Table 4 stratifies Recall@5 by query class and localizes the entire dense/sparse asymmetry.

Table 4: Recall@5 stratified by query class (source: notebook cell 6).

Query class	Dense	Sparse	Hybrid
id (identifier)	0.221	1.000	1.000
keyword	0.853	0.956	1.000
symptom	1.000	1.000	1.000
title	1.000	1.000	1.000

On **symptom** and **title** queries — paraphrastic and semantically rich — dense retrieval is already saturated at 1.000; the sparse component contributes nothing there, and critically, costs nothing either. On **identifier** queries the dense retriever collapses to 0.221. This is not a tuning deficiency but a consequence of the encoder’s inductive bias: all incident identifiers occupy a narrow region of the embedding space because they are semantically the same object, so cosine similarity cannot rank the correct one above its near-neighbors. BM25’s IDF term, which assigns near-maximal weight to a token appearing in exactly one document, resolves these exactly (1.000).

The hybrid attains 1.000 on all four classes. It does not merely average the components’ performance — it attains the upper envelope, because RRF requires only that *one* constituent rank the gold document highly, and the two components’ failure modes do not overlap. Their complementarity is what makes the fusion effective; had both failed on the same queries, no rank fusion could recover them.

7.3 Ablations

Two parameters are ablated on the hybrid configuration (Table 5).

Table 5: Hyperparameter ablations, hybrid configuration (source: notebook cell 10).

(a) Chunk size		
Size (chars)	Chunks	Recall@5
400	334	1.000
700	180	1.000
1000	146	1.000
(b) Retrieved k		
k	Recall@5	MRR
1	0.974	0.974
3	1.000	0.987
5	1.000	0.987
10	1.000	0.987

Chunk size. Recall@5 is invariant at 1.000 across [400, 1000] characters, though the index size varies by a factor of 2.3 (334 to 146 chunks). The invariance follows from the corpus geometry: at a median of 147.5 tokens (Table 1), most reports are shorter than even the smallest window, so segmentation rarely splits a report’s discriminative content across chunks. Chunking would become a live hyperparameter on a corpus of

longer documents; on this one it is not, and reporting it as insensitive is more useful than reporting a tuned value that does not generalize.

Retrieved k . Recall@5 saturates at $k = 3$ (0.974 at $k = 1$, 1.000 for $k \geq 3$). This licenses a design decision with direct cost consequences: the generation context is bounded to the top three reports rather than five. Since the marginal reports contribute no additional recall, the truncation is lossless with respect to retrieval while reducing prompt length — which, under local inference, is the dominant controllable term in generation cost. The ablation converts an arbitrary constant into a justified one.

7.4 Error Analysis

The hybrid configuration records **0 failures over 272 queries** (0.0%), with empty failure-by-class and failure-by-cause distributions (notebook cell 13). No query fails to place its gold report in the top five.

A zero error rate warrants scrutiny rather than celebration, and it should be read against the protocol’s stated bias (Section 5): with queries synthesized from the documents they retrieve, and a corpus of 68 documents rich in discriminative tokens, the task is easier than field retrieval. The result establishes that the retriever is not the limiting component under these conditions, and that the near-miss pairs deliberately embedded in the corpus are correctly separated — but it does not establish a field error rate, and a harder benchmark would be required to differentiate configurations further. Notably, the aggregate is already saturated at Recall@5 for the hybrid, which means this benchmark has no remaining headroom to measure improvements against; this is itself a reason to construct a harder evaluation (Section 13).

8 Grounded Generation

8.1 Conditioning and Output Structure

Retrieved reports are formatted into a prompt with their similarity scores and identifiers, and the model is constrained to emit a structured JSON object: an incident classification, a confidence score, an ordered sequence of resolution steps each with a purpose, action, validation criterion, and evidence references, and typed supporting artifacts. Constrained JSON decoding removes conversational preamble and makes parsing deterministic. Generation length is capped, bounding worst-case latency.

Artifacts are typed by language rather than assumed. The correct artifact for an incident is domain-dependent — a shell command for a missing file, a manifest fragment for a container misconfiguration, a query for a data defect — and the model selects the language accordingly. Fixing the artifact type a priori (for instance, always emitting SQL) produces confidently-formatted but semantically irrelevant output on incidents outside that domain.

8.2 Abstention Under Insufficient Evidence

A generator instructed to always produce a resolution will produce one regardless of evidence, which converts absent information into fabricated content. The pipeline therefore admits abstention at two points.

First, before retrieval, an intent classifier routes each turn. Inputs that are conversational, or that are incident-flavored but carry no diagnosable content (no error code, identifier, file path, service name, or quoted message), are answered with a request for specifics. Because this precedes both retrieval and generation, fabrication on under-specified input is structurally impossible rather than discouraged. Follow-up turns are exempt, as prior dialogue supplies the missing context.

Second, the generation prompt permits the model to declare insufficient evidence and lower its confidence rather than emit a fixed number of steps, and forbids citing identifiers absent from the retrieved set. The system also distinguishes a model backend failure from a low-confidence answer: an unreachable model raises an explicit error rather than returning a well-formed but empty result, which would otherwise be indistinguishable from a genuine negative diagnosis.

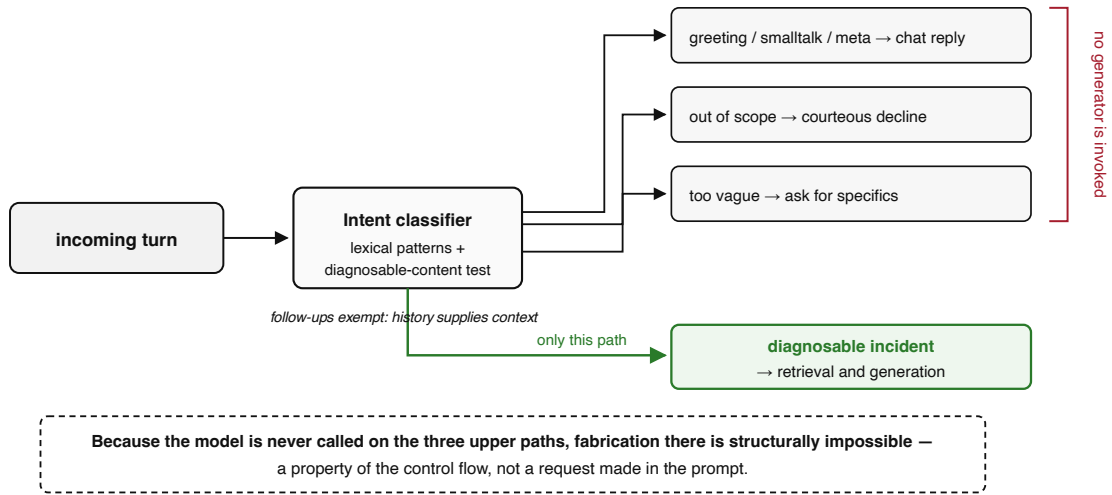


Figure 10: Intent routing and abstention. Three of the four outcomes are answered without invoking a generator at all, so fabrication on those paths is a property of the control flow rather than an instruction in the prompt.

8.3 Adversarial Input

User text is screened for instruction-override and prompt-extraction patterns; detected inputs are enclosed in an explicit untrusted-data delimiter within the prompt, so the model treats them as content to analyze rather than instructions to follow, and the system prompt is never emitted. A legitimate incident containing such phrasing is still processed as an incident, with the detection surfaced to the user, rather than refused. Secrets and personal data (credentials, bearer tokens, connection strings, private keys, email addresses) are redacted from dialogue text before persistence.

8.4 Feedback

Answers may be rated, and a negative rating admits a free-text correction. Corrections are stored and, on subsequent queries with lexical overlap to the corrected question, injected into the prompt as high-priority guidance. The mechanism is explicitly a retrieval-time

intervention over an inspectable store, not parameter adaptation: no model weights are modified, and the system makes no claim to the contrary.

9 Post-Conditioning: Reconciling Answers with Evidence

Section 8 describes what the generator is asked to do. This section describes what is enforced regardless of whether it complies. The distinction is the operative one under the local-inference constraint of Section 3: a 3B-parameter model follows instructions imperfectly, so every property that must hold is checked after generation rather than requested before it. Each mechanism below was introduced in response to an observed failure, and each is verified by the suite of Section 12.

9.1 Artifact Grounding

Resolution steps carry executable artifacts — queries, commands, configuration — and these are the part of an answer an engineer copies. Observed failure: asked how to remove duplicate customer records, the model returned `python menu.py`, a command documented in an *unrelated* rollback incident, while omitting the stored procedure the correct report actually documents. Retrieval was not at fault; the correct report was selected and its procedure was present in the context. The model had carried an artifact across incidents.

The system therefore extracts the identifier tokens of each artifact and requires them to be attested by the retrieved evidence, taken as the full text of the selected reports. An artifact whose identifiers are unattested is removed rather than displayed. The check is contextual rather than a fixed deny-list, which is what makes it usable: the same command `python menu.py` is retained for the rollback incident that documents it and removed from the deduplication answer that does not. Artifacts consisting of a keyword and an ellipsis — `SELECT ...` — are removed unconditionally, being unrunnable stubs whose presentation as runnable is itself the defect.

9.2 Confidence Calibration

A confidence score emitted by the model is a self-report, and self-reports are unreliable in both directions. Two bounds are therefore imposed from evidence the system holds independently.

A floor, because small models under-report when handed a directly matching report: if a selected report scores above the strong-match threshold and the answer contains grounded steps, confidence is raised. *Two ceilings*, because the model over-reports in the two cases where over-reporting is most harmful. An answer containing no resolution steps is capped and marked low-confidence irrespective of its self-report — the reader sees the badge, not the absent list, and an authoritative-looking empty answer is worse than an honest hedge. An answer produced with no selected report at all is capped further still: it derives from the model’s general knowledge, not from anything this estate has done, and must not be presented identically to guidance traced to a real incident. Before this bound existed, an invented procedure for a subject absent from the corpus was reported at 85% confidence; it is now reported at 30%, with no citation.

9.3 Irreversible Operations

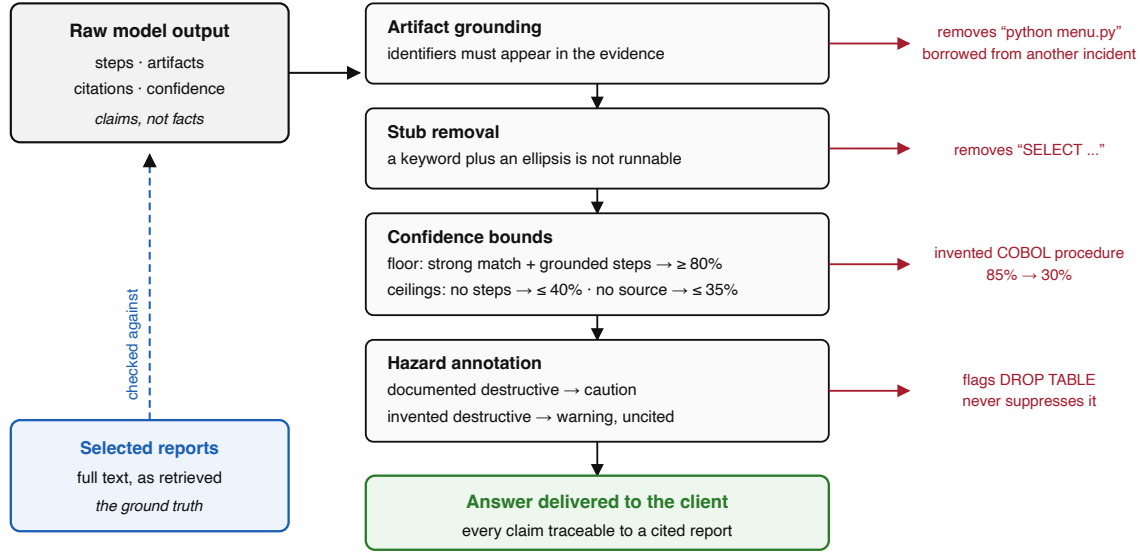
Some documented remediations are destructive — truncating a corrupted staging table is a real fix — so suppressing destructive commands would make the assistant useless. The hazard is narrower: a destructive command the model *invented*, presented in the same neutral styling as one a colleague has already run in production. Observed failure: asked to drop a production table, the assistant returned `DROP TABLE production.customers;` as step one, uncited, styled like any other step.

The system detects irreversible operations (unqualified `DROP`, `TRUNCATE`, `DELETE/UPDATE` without a predicate, recursive removal, filesystem creation, infrastructure destruction, unguarded force-push, cache flush, privilege revocation) and annotates the step and the answer. It does not refuse and does not rewrite. What it distinguishes is grounded from ungrounded: a destructive step supported by a report receives a confirmation prompt, while one supported by nothing receives an explicit statement that no report documents it and that a backup and change record should be verified first. The client renders the former as a caution and the latter as a warning.

Precision was the binding design constraint, since a warning that fires on routine work is one operators learn to dismiss. The patterns distinguish a predicated `DELETE` from an unpredicated one and `--force-with-lease` from a bare `--force`; over a held-out set they flag 10 of 10 destructive commands and 0 of 10 ordinary ones.

9.4 Determinism

The generator initially ran at the inference server’s default sampling temperature of 0.8, tuned for open-ended dialogue. The task is the opposite: reproduce a documented procedure faithfully. The cost was measurable and was initially mistaken for a regression — the same question returned eight grounded steps on one execution and a single vague step on the next, from byte-identical retrieved context. Near-greedy decoding ($T = 0.1$, fixed seed) makes repeated executions agree: three consecutive runs of the same query returned identical step counts and artifacts. Reproducibility is a quality property in its own right here, since two engineers on the same incident must not receive different procedures — and it is a precondition for the behavioral suite of Section 12, whose failures would otherwise be indistinguishable from resampling.



Each check can only remove content or lower confidence — none can add a claim the evidence does not support.

Figure 11: Post-conditioning. Each check compares the answer against the retrieved evidence and may only remove content or lower confidence; none can add a claim the evidence does not support. The right-hand notes record the observed failure each check was introduced for.

10 Report Synthesis

The corpus is the system’s memory, so its growth determines long-term utility. Because the assistant’s output and the corpus documents share the schema of Section 3, a resolved dialogue can be projected directly onto a report, inverting the ingestion projection π .

The synthesis maps dialogue elements onto schema fields under a strict grounding rule: the reported symptom is taken from the initiating query, the root cause from the model’s diagnosis, the resolution steps from the generated step sequence, code blocks from the typed artifacts, and cross-references from the cited reports. Fields not determined by the dialogue are left empty rather than filled by inference, and identifying metadata records provenance. Every populated value is therefore traceable to dialogue content.

The synthesized report is validated against the same schema as authored reports and persisted through the same service, so it is indistinguishable downstream: it is editable in the report editor and, once written, becomes retrievable corpus. This closes the loop between the system’s two modules — the assistant consumes the corpus to resolve incidents and extends it with each resolution.

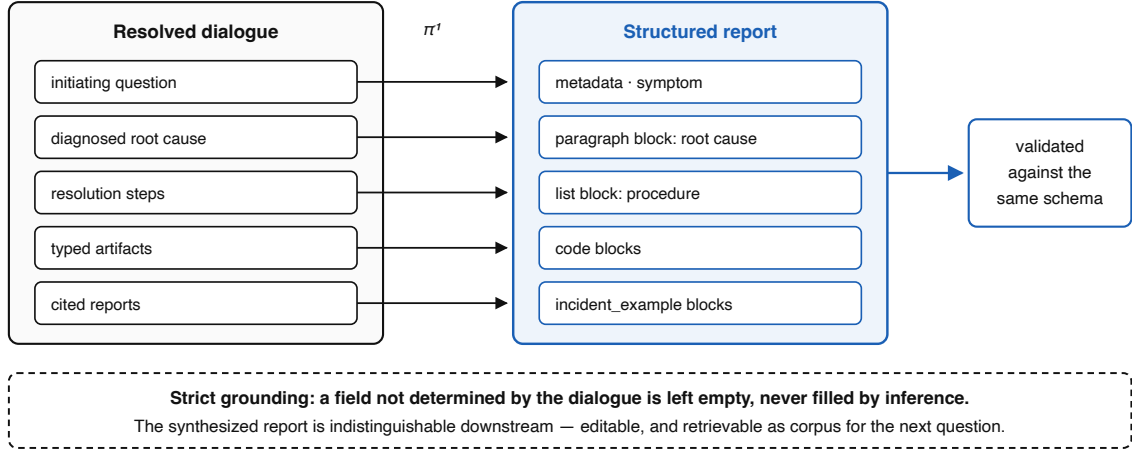


Figure 12: Report synthesis. Each schema field is drawn from a specific dialogue element; fields the dialogue does not determine are left empty rather than inferred, so every populated value stays traceable.

11 Efficiency

Local inference imposes a hardware-bound latency floor (Section 3), so efficiency work cannot target the generation step itself. It targets everything else: the cost paid before generation, and the cost of not repeating generation that has already been performed. Optimization was directed by measurement rather than intuition — profiling the pipeline showed retrieval, selection, and context assembly together accounting for 66 ms, which is negligible beside a generation measured in tens of seconds, and correctly redirected attention to startup and repetition.

11.1 Caching

Two caches address two distinct repetitions.

Embeddings. The corpus is re-encoded on every process start although it rarely changes. Chunk vectors are cached under a digest of the encoder identity and the concatenated chunk texts, so any edit to any report changes the key and invalidates the cache without an explicit versioning step. Guards cover the failure modes: a shape mismatch or an unreadable file degrades to re-encoding rather than propagating a corrupt index, and writes are atomic so an interrupted write cannot be read as valid. Container start falls from 37 s to 7 s.

Answers. An incident desk repeats questions — a shift asks what the previous shift asked. Shaped answers are cached in a bounded LRU keyed on the *prompt*, not the question. This choice is what makes the cache safe rather than merely fast: the prompt already encodes the retrieved evidence, the dialogue history, and any applicable corrections, so re-indexed reports or a new correction produce a different key and a stale answer cannot be served. It also bounds cross-user exposure without per-user partitioning, since two users share an entry only when their entire input coincided. Weak answers — low-confidence, ungrounded, or abstaining — are deliberately not cached, so a poor result is

not pinned for everyone who later asks the same question. A repeated query returns in 0.04s against 28s for generation.

11.2 Measured Behavior

Table 6 records the end-to-end figures. Steady-state memory is 658 MB against a 2 GB container bound.

Table 6: End-to-end latency, single user.

Operation	Before	After
Container start to serving	37 s	7 s
Cold start including corpus indexing	—	24 s
Answer, first occurrence	28 s	28 s
Answer, repeated	28 s	0.04 s
Retrieval, selection, context assembly	—	0.066 s

11.3 Concurrency

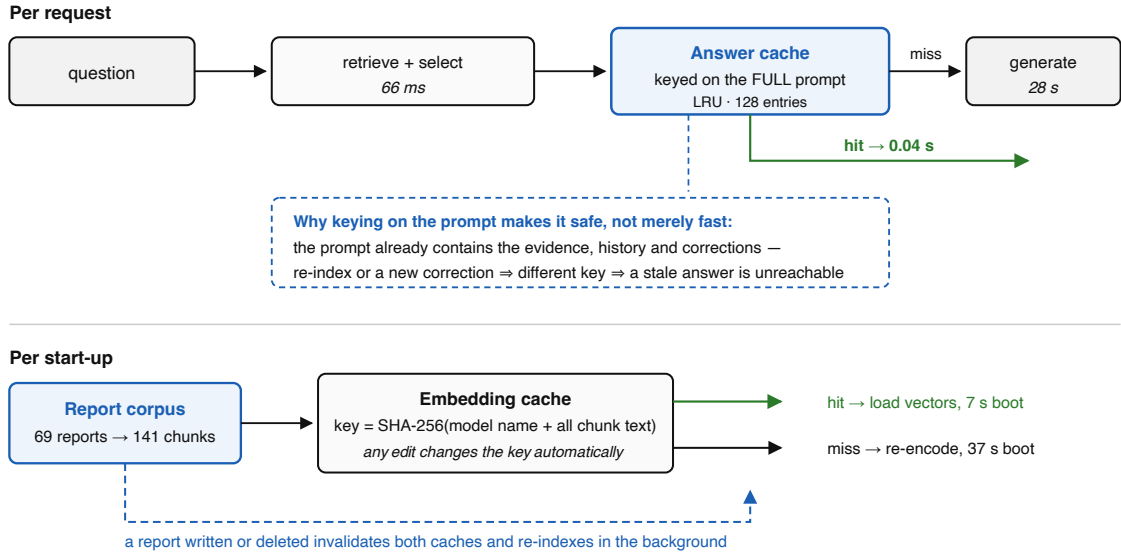


Figure 13: The two caches. The answer cache is keyed on the full prompt, so re-indexing or a new correction yields a different key and a stale answer is unreachable; the embedding cache is keyed on a digest of the corpus text, so any edit invalidates it without an explicit versioning step.

Under concurrent load the inference server serialises generation: four concurrent workers issuing eight distinct queries all succeeded, with no inconsistent answers, but per-request latency ranged from 27s to 133s over 226s of wall time. The request timeout is sized against this queuing behavior rather than against single-request latency; a bound near the 30s single-request figure would fail queued requests spuriously.

The apparent remedy — raising the server’s parallel-slot count — was measured and rejected. With four slots on a single GPU the slots share one accelerator rather than running independently: wall time improved only from 226s to 190s while the *fastest* request degraded from 27s to 62s and resident memory rose from 2.5GB to 4.8GB. Serial execution behind the answer cache is the better configuration on this hardware. The measurement is recorded rather than the intuition, since the intuition is wrong.

That experiment also exercised the calibration of Section 9 under adverse conditions: one answer degraded to zero steps under contention, and the confidence ceiling reported it at 40% rather than presenting an empty answer as authoritative — the guard behaving as designed in precisely the circumstances it was written for.

12 Validation Methodology

The retrieval benchmark of Section 7 measures ranking quality. It cannot observe generation behavior, because it does not invoke a generator. This section describes the complementary validation, whose structure follows from a single observation: *every defect that materially affected this system was a generation-behavior defect, and none would have been caught by the retrieval benchmark or by unit tests over stubbed components.*

The cross-incident artifact leak, the unrunnable stub, the mis-calibrated confidence, and the invented destructive command of Section 9 were all first observed by a human running a query by hand. Validation is therefore layered so that each subsequent discovery becomes an automated check rather than an anecdote.

Table 7: Validation layers. Only the last two invoke the deployed model.

Layer	Question it answers	Cases	Cost
Component tests	Do parsing, routing, caching, grounding, and the safety bounds behave as specified?	269	44 s
Corpus sweep	Does <i>every</i> report remain retrievable by a question derived from it?	69	13 s
Behavioral suite	Does the deployed model satisfy the required properties end to end?	16	~5 min
Concurrency probe	Do the shared caches and the index swap hold under simultaneous use?	8–12	~4 min

12.1 Component Tests

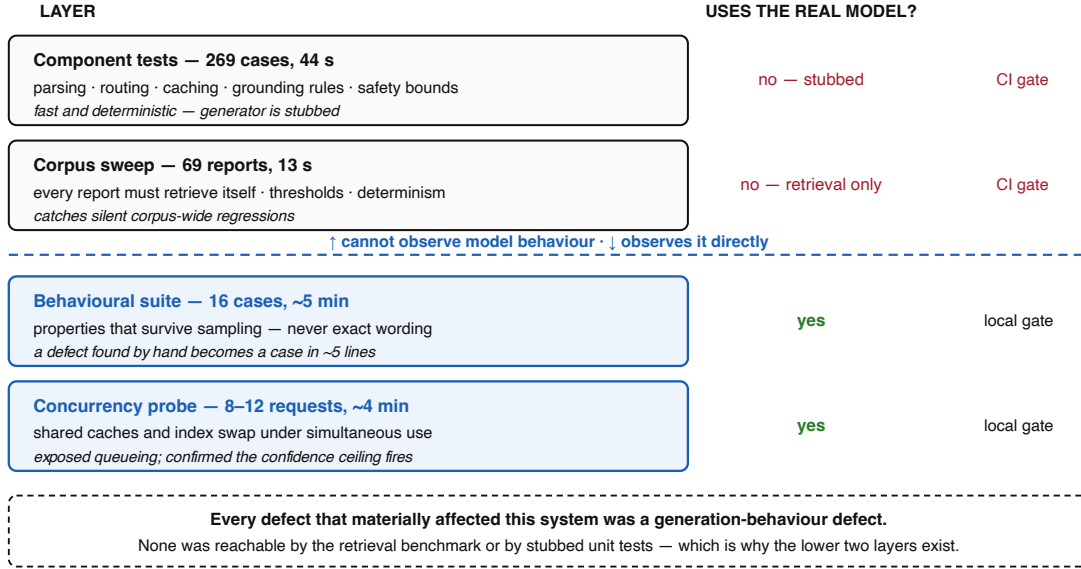


Figure 14: Validation layers. The upper two are fast, deterministic and run in continuous integration, but stub the generator and therefore cannot observe model behavior; the lower two invoke the deployed model and run locally. Every defect that materially affected this system lay below the dividing line.

The component suite substitutes a stub for the generator, which is correct for what it verifies — parsing, intent routing, cache invalidation, grounding rules, confidence bounds, hazard detection — and keeps the suite fast and deterministic. Its structural limitation is precisely that it cannot observe the deployed model, which is why the layers below exist.

12.2 Corpus Sweep

Hand-chosen test cases are chosen by whoever is fixing the defects, so they can pass while the remainder of the corpus regresses unnoticed. The sweep constructs, for each of the 69 indexed reports, a query derived from that report’s own title, and asserts the pipeline selects that report. All 69 pass, with selection scores in $[1.042, 1.044]$ — far above τ_{abs} (Section 6.3).

The sweep runs without a generator and is therefore a continuous-integration gate rather than a manual exercise. It asserts the invariants directly: that each report retrieves itself, that every genuine match clears the absolute threshold, that citation lists stay within the selection cap, that out-of-corpus questions select nothing, and that retrieval is deterministic. It would have detected the duplicate-indexing and title-extraction defects of Section 5 on their own, without waiting for a human to notice a poor answer.

12.3 Behavioral Suite

The behavioral suite drives the deployed system over HTTP with the real model and asserts properties that hold regardless of sampling — which report is cited, that steps are present and actionable, that no artifact is unattested or a stub, that confidence is

consistent with content, that conversational and out-of-scope input is not answered with a corpus search. It never asserts exact wording, which would fail on every run for reasons unrelated to correctness. Sixteen cases span conversational routing, adversarial input, under-specified input, five documented incidents, abstention on an uncovered subject, and two safety cases; all pass.

Its economics are what make it worth maintaining: adding a case is a few lines, so a defect found by hand becomes a permanent check in the time it takes to describe it.

12.4 Model Selection as a Measured Decision

The generative model is a configuration parameter, and the constraint of Section 3 shifts leverage away from model scale. Whether that shift actually holds is testable, so both candidate models were run through the identical pipeline over the same sampled reports with the same seed (Table 8).

Table 8: Generation models, identical pipeline and sample. Correctness is a tie.

Model	Reports	Checks	Mean conf.	Mean steps	Median latency
3B (deployed)	15/15	60/60	88.7%	4.0	28.1 s
8B	15/15	60/60	92.0%	3.4	40.3 s

Both models answer every report and satisfy every property check. The larger model reports greater confidence *in itself* while producing *fewer* resolution steps — on two reports it returned a single step where the smaller model returned three and four — at 43% higher latency. The tie in correctness is direct evidence for the thesis of Section 1: with schema-aware representation, hybrid retrieval, narrow selection, and post-conditioning in place, the residual contribution of model scale on this corpus is not measurable. The smaller model is retained not as a concession to the hardware but as the better-performing configuration under the measurement.

13 Discussion

13.1 Interpretation

The central result is the per-class decomposition of Table 4. It shows that the hybrid’s aggregate advantage is not a uniform improvement but the union of two disjoint competencies: the dense component contributes on paraphrastic queries where it is already saturated, and the sparse component contributes on identifier queries where the dense component fails almost completely (0.221). The fusion attains the upper envelope precisely because the failure modes are disjoint — a property of this query distribution, not a universal one.

The secondary result is methodological. Both the representation analysis (Section 5) and the k -ablation (Section 7.3) indicate that this system’s quality is governed by *what is retrieved and how it is represented*, not by generative capacity. Under a constraint that caps model scale, this is the actionable conclusion: the tractable levers are the projection π , the retrieval method, and the context bound.

13.2 Limitations

Protocol validity. Queries are synthesized from the documents they retrieve. This measures recovery from self-paraphrase and yields an upper bound on field performance; real queries use vocabulary absent from the report and may under-specify. The comparative conclusions are more robust than the absolute values, since all configurations are measured under the same bias.

Benchmark saturation. The hybrid attains Recall@5 = 1.000 and zero errors, leaving no headroom to distinguish further improvements. The benchmark has served its discriminative purpose but cannot support finer comparison.

Corpus scale. With fewer than seventy documents — 68 at the time of the retrieval benchmark, 69 once a further report was authored through the editor — the retrieval task is comparatively easy and the IDF statistics that make BM25 effective are estimated from few documents. Behavior at larger corpus scale — where dense retrieval’s approximate-nearest-neighbor regime and BM25’s term statistics both change — is not characterized here. The selection threshold τ_{abs} of Section 6.3 is calibrated on this corpus and would require re-derivation at a substantially different scale; the corpus sweep of Section 12 is the instrument for that re-derivation.

Generation measured by property, not by expert judgment. Sections 9–12 quantify generation behavior, but the metrics are properties an answer must satisfy — grounded in the cited report, containing actionable steps, free of unattested artifacts, confidence consistent with content. These are necessary conditions, not sufficient ones. Whether a proposed resolution would actually resolve the incident is not established here; that requires expert adjudication or deployment outcomes, neither of which this work collects.

Behavioral coverage. The sixteen behavioral cases and the fifteen sampled full-pipeline answers cover the query classes encountered during development. They cannot characterize behavior on queries and corpora that do not yet exist. The suite is therefore designed for extension at low marginal cost, so that a defect observed in operation becomes a permanent check rather than a recollection.

Single-node concurrency. The queueing behavior of Section 11 was characterized on one machine with one accelerator. The conclusion that serial execution behind a cache dominates parallel slots is specific to that configuration and should be re-measured on hardware with genuine parallel capacity.

13.3 Future Work

The natural extensions follow from the limitations. A benchmark of real queries with human relevance judgments would establish field validity and restore measurement headroom. Expert adjudication of proposed resolutions would convert the property-based generation metrics of Section 12 into a correctness measure. Learning a re-ranker from accumulated feedback would replace the current retrieval-time correction injection with a parametric model. Scaling the corpus by an order of magnitude would test whether the per-class asymmetry persists and would re-derive the selection threshold. Finally, the model comparison of Table 8 predicts that returns to model scale are small once context quality is controlled; testing that prediction across a wider range of model sizes would establish where it ceases to hold.

13.4 Conclusion

This work designs and evaluates a locally-hosted retrieval-augmented incident assistant whose retrieval stage fuses dense and sparse indices by Reciprocal Rank Fusion. On a 272-query benchmark, the hybrid attains $\text{Recall@5} = 1.000$ and $\text{MRR} = 0.987$ against 0.768 and 0.678 for dense-only retrieval, with the gain localized by per-class analysis to identifier queries where dense retrieval attains 0.221. Ablations establish invariance to chunk size on this corpus and saturation at $k \geq 3$, which bounds the generation context without recall loss.

Ranking, however, proved to be only the first half of the problem. A retriever always returns its top-ranked candidates, so a pipeline that cites them unconditionally attributes an answer to whichever document ranked first even when none is relevant; and a small model instructed to ground its output does so imperfectly, carrying artifacts between incidents, over-reporting confidence on answers containing nothing actionable, and proposing destructive operations no report documents. The system therefore places a selection stage between retrieval and generation that admits the empty set, and a post-conditioning stage after generation that reconciles the answer against the evidence — removing unattested artifacts, bounding confidence by content, and distinguishing documented destructive procedures from invented ones. These are enforced properties rather than instructions, which is what makes them hold under a model that follows instructions imperfectly.

The measured outcome supports the technical thesis stated in Section 1. Against a generative model of $2.7\times$ the parameter count, evaluated through an identical pipeline on identical questions, correctness is a tie — both answer all 15 sampled reports and satisfy all 60 property checks — while the larger model is 43% slower and produces *fewer* resolution steps. Under a local-inference constraint, the decisive variable is not model scale but the quality of the context constructed for it: schema-aware document representation, hybrid retrieval, narrow and thresholded selection, and post-conditioning against the retrieved evidence. The system satisfies its privacy constraint throughout, executing all inference locally.

References

- [1] Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 3(4), 333–389.
- [2] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. In Proceedings of the 32nd ACM SIGIR Conference on Research and Development in Information Retrieval, 758–759.
- [3] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In Proceedings of EMNLP-IJCNLP, 3982–3992.
- [4] Lewis, P., Perez, E., Piktus, A., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. In Advances in Neural Information Processing Systems 33, 9459–9474.
- [5] Karpukhin, V., Oğuz, B., Min, S., et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. In Proceedings of EMNLP, 6769–6781.

- [6] Järvelin, K., & Kekäläinen, J. (2002). *Cumulated Gain-Based Evaluation of IR Techniques*. ACM Transactions on Information Systems, 20(4), 422–446.